# Features and choosing a model, and your project

*Which model should we recommend for predicting crusher energy demand, and how do we explain the choice to someone who pays?*

CML3113 | Mining and Petroleum Engineering | Week 11, class 4 of 4 | Friday 27 November 2026, 08:00 to 11:00

KLAi | International University of East Africa | Jotham Martin Wambi

# Today in one view: what you will leave with

## By the end of today you can



Build a new feature, test it with a feature ablation (ablation study) and say what the result shows



Compare five candidate models with the same cross-validation protocol (model selection) and choose the simplest one that is good enough (parsimony)



Write a one-page decision memo and explain how the timed, auto-graded project works

# Your three hours: models and memo first, then the lab

| | | |
|---|---|---|
| 0:00 | 10 | Recall from Class 3 |
| 0:10 | 35 | Concepts, part 1: features, scaling and two new models |
| 0:45 | 30 | Concepts, part 2: a fair comparison, the memo and the project |
| 1:15 | 15 | Break |
| 1:30 | 55 | Lab: the model comparison table, in the notebook |
| 2:25 | 25 | Open and explain the Block 2 quiz (open 24 hours) |
| 2:50 | 10 | Close and the project briefing |

Concepts first, with no laptops. Then the lab. The quiz opens in the last 20 minutes (open 24 hours).

# Three questions from last time

1. What is overfitting, and how do we spot it?
2. What is K-fold cross-validation for?
3. What is data leakage?




Answer aloud first. Then we check together.

# Words we use today

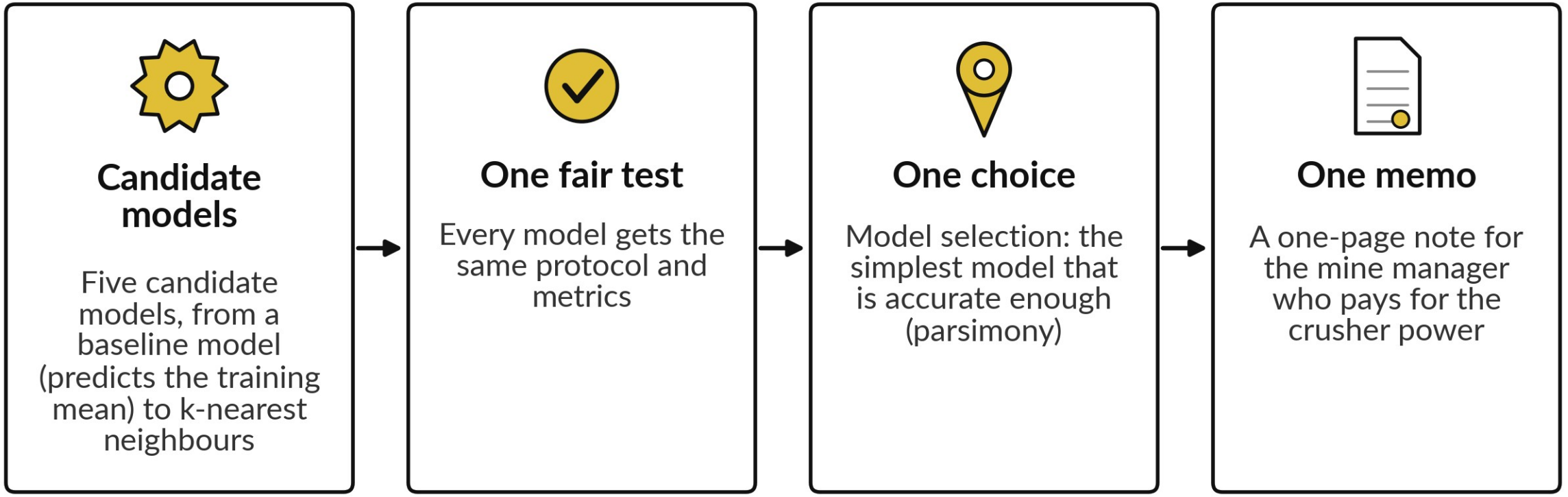
| | Word | What it means |
|---|---|---|
|  | **Feature engineering** | Building a new feature (input variable) from existing features, using what you know about the machine. |
|  | **Feature ablation** | Also called an ablation study. Train with a feature and without it, on the same rows and metric, and compare the errors. |
|  | **Ridge regression** | Linear regression with regularisation: an L2 penalty on large weights. Alpha is the regularisation strength. |
|  | **k-NN** | k-nearest neighbours: average the targets of the k closest training rows. k is a hyperparameter (a setting chosen before training). |
|  | **Decision memo** | A one-page note with your choice, numbers and limits. A related industry term is model card. |

# Class 3 in short: overfitting, cross-validation and data leakage

| | Question from Class 3 | Short answer |
|---|---|---|
|  | **What is overfitting, and how do we spot it?** | The model fits noise in its training set, so it does well on training rows but worse on new rows. We spot it by a large generalisation gap: small training error, much bigger validation error. |
|  | **What is K-fold cross-validation for?** | K-fold cross-validation tests a model several times. Each time a different fold (part) of the training set is held out for scoring, so we see how much the score moves. |
|  | **What is data leakage?** | Information from the held-out rows reaching the model while it learns, for example a mean worked out on all rows. The score looks too good. |

Check your own answers against these before we move on to today's ideas.

# In real work you compare several models, choose one and explain why




The picture shows today's four moves. Whoever pays for the crusher work wants the choice, the error in kWh/t and the doubts on one page.

# A new feature can carry what you know about the machine

| | A | B | C | D | E |
|---|---|---|---|---|---|
| | record_id | load_pct | ambient_c | load_x_ambient | target |
| 1 | 1 | 59.15 | 31.71 | 1875.6 | 14.0 |
| 2 | 2 | 77.46 | 29.97 | 2321.5 | 15.1 |
| 3 | 3 | 69.6 | 19.87 | 1383.0 | 13.0 |

**Three training rows. The highlighted column is a new feature: load_pct times ambient_c.**

Feature engineering builds a new feature from existing ones. Here, hot air may change what a load costs.

A product of two features is an interaction feature. Row 1: 59.15 x 31.71 = 1875.6.

# The new feature is a product, and the extended linear regression model gets one more weight

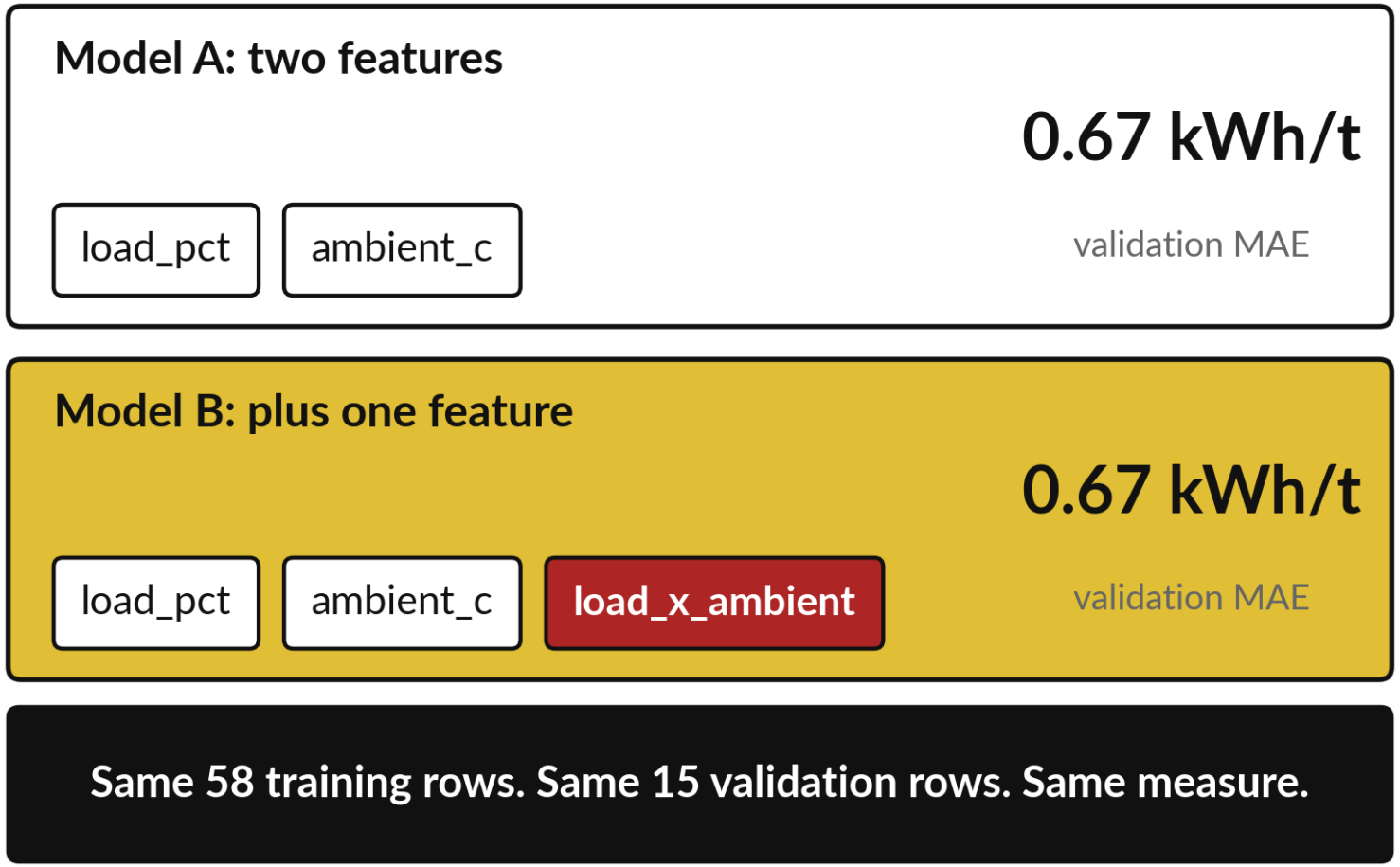
New column $$x_3 = x_1 \times x_2$$

Model A, multiple linear regression $$\hat{y} = 1.1568 + 0.1201\,x_1 + 0.1587\,x_2$$

Model B, plus interaction feature $$\hat{y} = 1.9344 + 0.1074\,x_1 + 0.1276\,x_2 + 0.0005\,x_3$$

| Symbol | What it means | In our data |
|---|---|---|
| **$x_1$, $x_2$** | load_pct (percent), ambient_c (deg C) | Row 1: 59.15 and 31.71 |
| **$x_3$** | load_x_ambient: $x_1$ times $x_2$ | Row 1: 1875.6 |
| **y-hat** | Prediction (kWh/t) | Row 1, B: 1.9344 + 6.35 + 4.05 + 0.94 = 13.3 |
| **$b_0$** | Intercept (bias): y-hat when every feature is 0 | A: 1.1568  B: 1.9344 |
| **$b_1$, $b_2$, $b_3$** | Weights (coefficients): one more unit of a feature adds its weight | A: 0.1201, 0.1587.  B: 0.1074, 0.1276, 0.0005 |

# A feature ablation tests one change on its own, with everything else kept the same

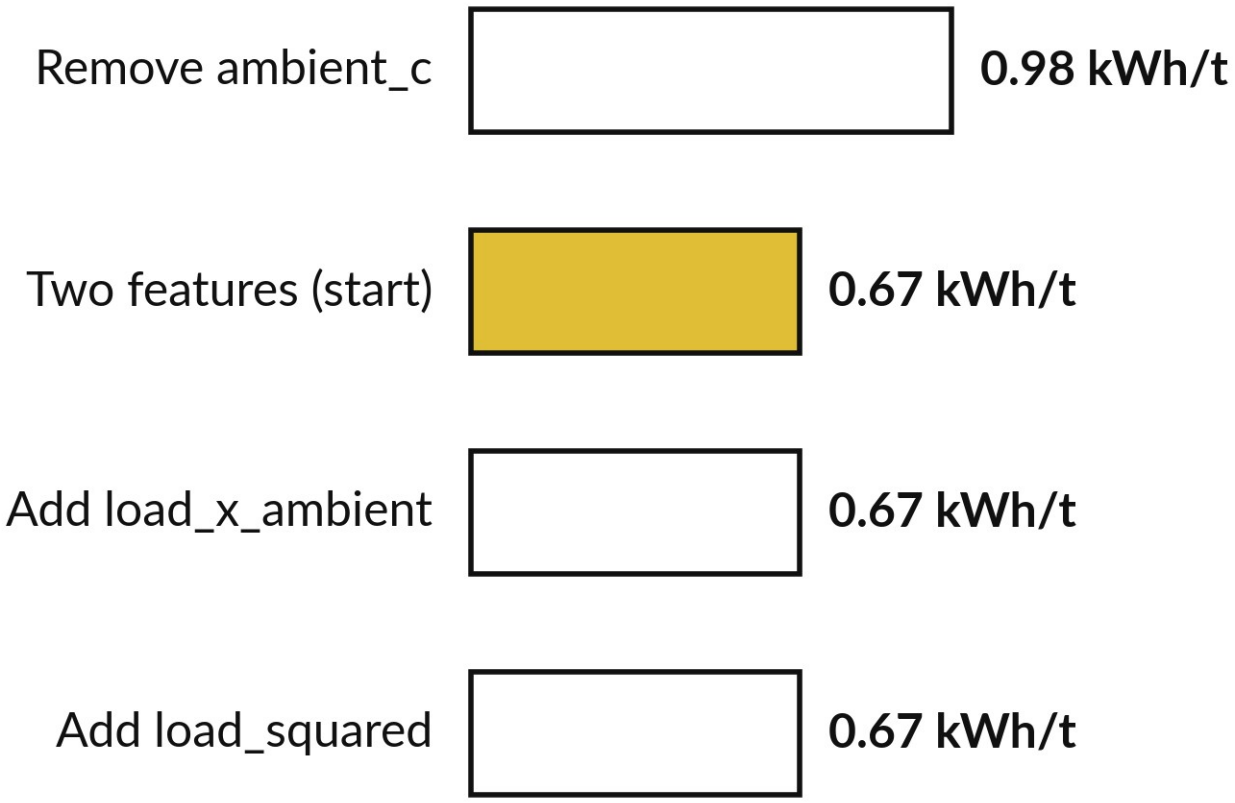



**A feature ablation (ablation study): train with a feature and without it, then compare the errors.**

Red box: the new feature. MAE is the mean absolute error, the average size of the prediction error.

Same training and validation sets. Only the features differ.

# Removing ambient_c hurts a lot, and the new features do not help




**Each bar is the validation MAE of one version of the linear regression model. A shorter bar is a better model.**

Without ambient_c the error grows from 0.67 to 0.98 kWh/t.

Adding load_x_ambient or load_squared (load times itself) changes it by almost nothing.

Test every idea before you keep it.

# One-hot encoding turns a categorical feature into 0 and 1 columns, and equipment_id stays out of the features

| | A | B | C | D | E |
|---|---|---|---|---|---|
| | **equipment_id** | **is_E1** | **is_E2** | **is_E3** | **is_E4** |
| 1 | E1 | 1 | 0 | 0 | 0 |
| 2 | E2 | 0 | 1 | 0 | 0 |
| 3 | E3 | 0 | 0 | 1 | 0 |
| 4 | E4 | 0 | 0 | 0 | 1 |
| 5 | E5 (new machine) | 0 | 0 | 0 | 0 |

**One-hot encoding gives each category of a categorical feature its own column: 1 if the row is in it, 0 if not.**

The model trained on E1 to E4 has no is_E5 column, so E5 in these four columns is all zeros.

A column per known machine says nothing about a new one, so equipment_id stays out of the features.

# Scaling puts every feature on the same ruler

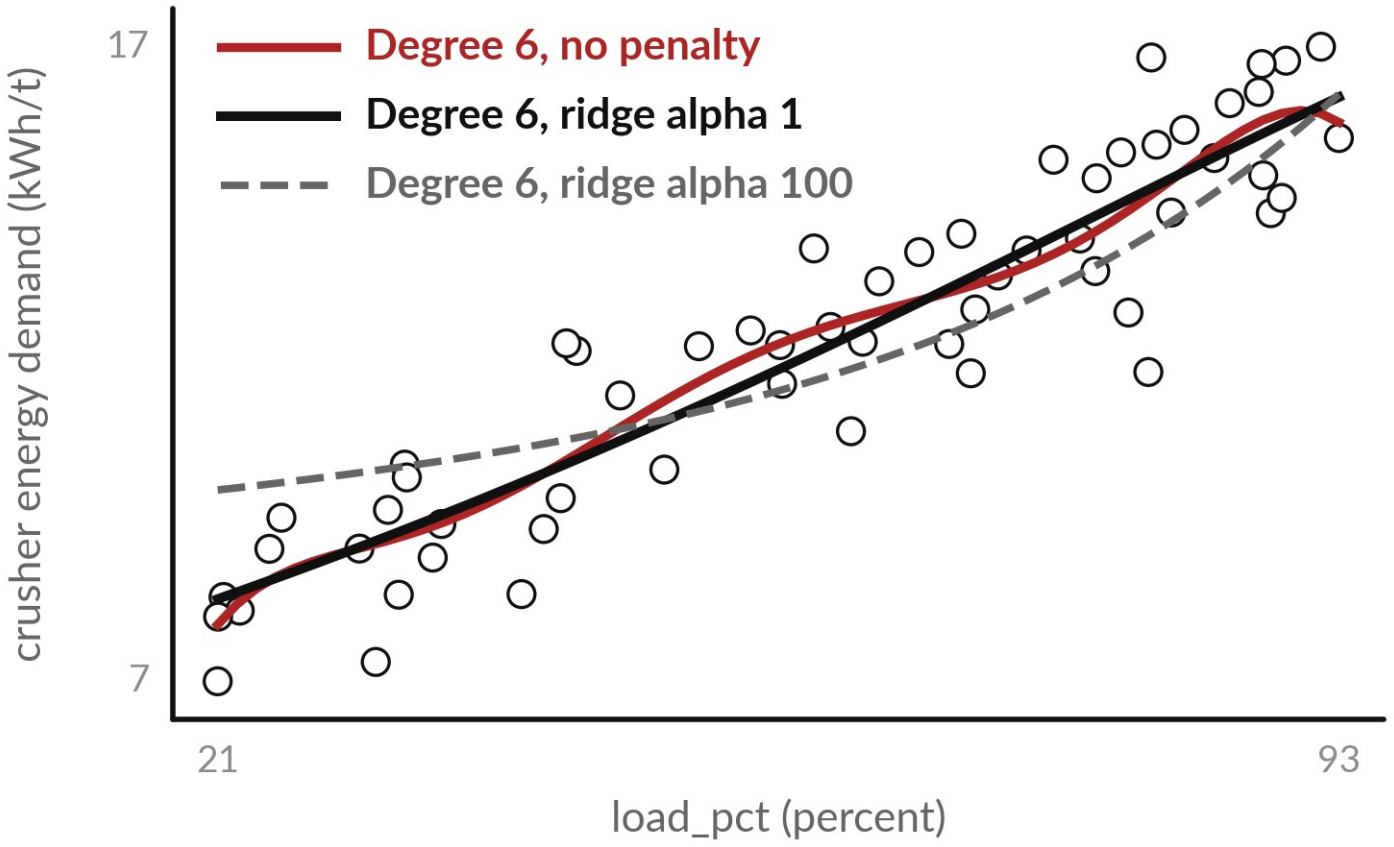
| | A | B | C | D |
|---|---|---|---|---|
| | load_pct | ambient_c | load scaled | ambient scaled |
| 1 | 59.15 | 31.71 | -0.03 | 1.18 |
| 2 | 77.46 | 29.97 | 0.77 | 0.80 |
| 3 | 69.6 | 19.87 | 0.43 | -1.42 |

**Standardisation (z-score): scaled value = (value minus training mean) divided by training standard deviation.**

Row 1 by hand: (59.15 - 59.9) / 22.7 = -0.03.

StandardScaler is the scikit-learn tool that learns the mean and standard deviation from the training set, then applies them to any rows. Ridge regression and k-NN need it.

# Ridge regression adds an L2 penalty on large weights so a flexible curve stops fitting noise




**Regularisation shrinks the weights (coefficients). Ridge regression adds an L2 penalty: alpha times the sum of squared weights.**

Dots: training rows. Curves: degree-6 fits of load_pct. A bigger alpha (the regularisation strength) gives a flatter curve.

Validation MAE (kWh/t): 0.98 none, 0.99 alpha 1, 1.48 alpha 100. Not the later two-feature model.

# Ridge regression minimises squared errors plus alpha times squared weights

Ridge loss

$$L = \sum_{i=1}^{n} (y_i - \hat{y}_i)^2 + \alpha \sum_{j=1}^{p} b_j^2$$

Prediction

$$\hat{y} = b_0 + b_1 z_1 + b_2 z_2$$

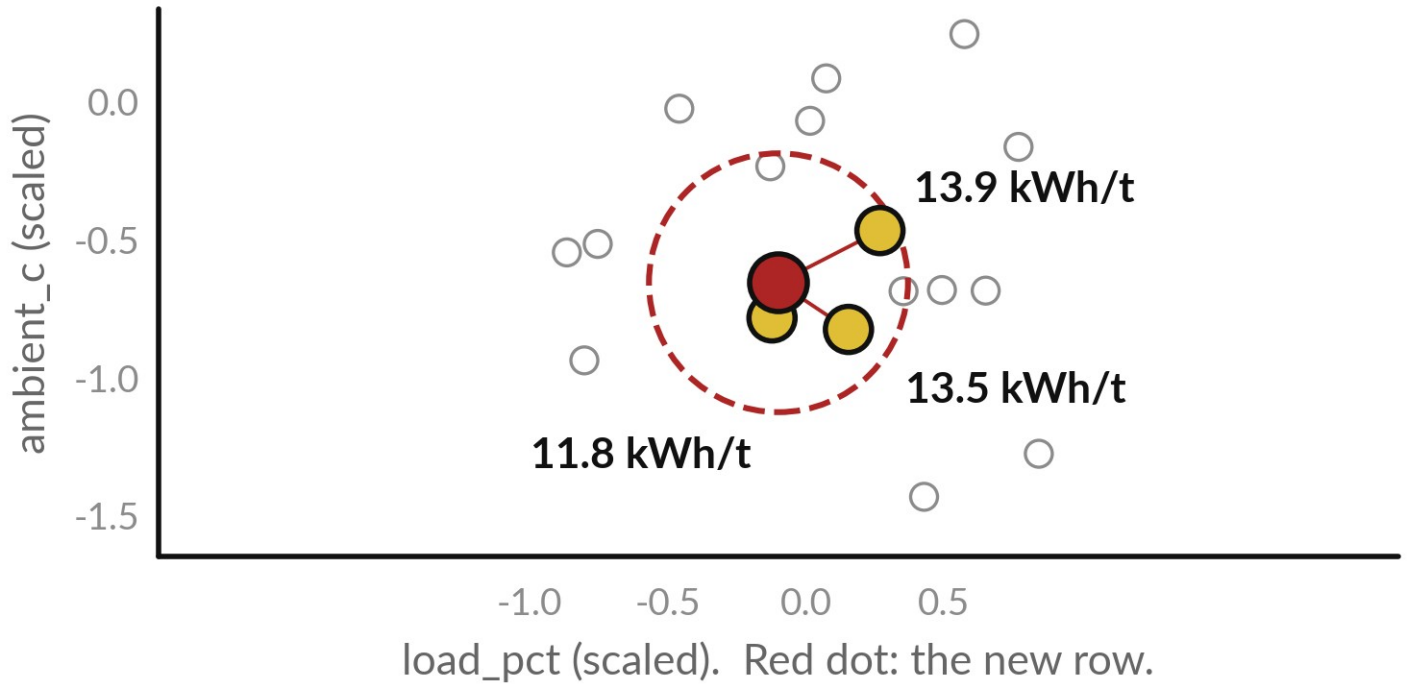
| Symbol | What it means | In our data |
|---|---|---|
| **L** | Loss function: the number made small | One value per choice of weights |
| **$y_i$, $\hat{y}_i$** | Actual and predicted target of training row i (kWh/t) | i = 1 to 58 |
| **n** | Number of training rows | n = 58 |
| **alpha** | Regularisation strength. 0 gives linear regression | We try 0.1, 1 and 10 |
| **$b_1$, $b_2$, p** | The p weights, one per feature. $b_0$ is not penalised | $b_1$ load, $b_2$ ambient, p = 2 |
| **$z_1$, $z_2$** | The features after scaling | $z_1$ scaled load_pct, $z_2$ scaled ambient_c |

# A bigger alpha shrinks the weights, but on this ridge regression on two features the error grows

| alpha | Weight $b_1$ (load) | Weight $b_2$ (ambient) | Sum of squared errors | L2 penalty: alpha x sum of squared weights | Loss L | Validation MAE (kWh/t) |
|---|---|---|---|---|---|---|
| **0 (ordinary linear regression)** | 2.73 | 0.72 | 35.5 | 0.0 | 35.5 | 0.67 |
| **0.1** | 2.72 | 0.72 | 35.5 | 0.8 | 36.3 | 0.68 |
| **1** | 2.68 | 0.70 | 35.7 | 7.7 | 43.4 | 0.70 |
| **10** | 2.31 | 0.57 | 46.1 | 56.6 | 102.7 | 0.90 |

Ridge regression on two features. load_pct was rescaled, so $b_1$ = 2.73 means: 22.7 more percent of load adds 2.73 kWh/t. Alpha 1: loss 35.7 + 7.7 = 43.4. Its error 0.70 is not the degree-6 curve error.

# k-nearest neighbours (k-NN) predicts from the k training rows most like the new row




**Prediction: average of the 3 = 13.1 kWh/t**

Actual value: 12.6 kWh/t

**k-NN stands for k-nearest neighbours: find the k training rows closest to the new row and average their targets.**

Grey dots: training rows. Red dot: the new row, record 5. Gold dots: its 3 nearest neighbours, labelled with their targets.

k = 3 is a hyperparameter (a setting chosen before training). Both axes are scaled.

# k-NN measures distance on the scaled features, then averages the k nearest targets

Distance

$$d = \sqrt{\sum_{j=1}^{2} (x_j - x'_j)^2}$$

Prediction

$$\hat{y} = \frac{1}{k} \sum_{i \in N_k} y_i$$

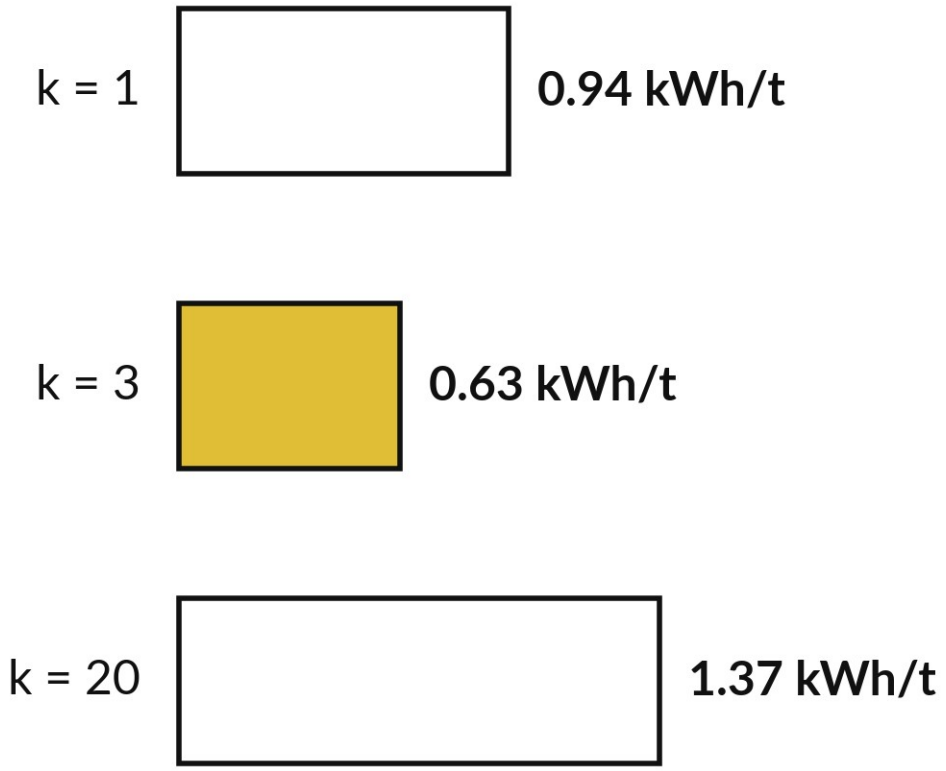
| Symbol | What it means | In our data |
|---|---|---|
| **d** | Distance from the new row to one training row | Nearest row: 0.129 (no unit) |
| **x, x′** | Scaled feature: new row (x), training row (x′) | Two features: scaled load_pct, scaled ambient_c |
| **k** | How many nearest rows we average. We choose it | k = 3 |
| **N** | The k training rows with the smallest d | Records 67, 22, 45 |
| **y** | Target of one neighbour (kWh/t) | 11.8, 13.5, 13.9 |
| **y-hat** | The prediction for the new row (kWh/t) | 13.1 kWh/t |

# Worked example: the four closest training rows to the new row, record 5

| | Training row (record_id) | Scaled load $x_1$ | Scaled ambient $x_2$ | Distance d | Target y (kWh/t) |
|---|---|---|---|---|---|
| 📍 | **67** | -0.123 | -0.770 | 0.129 | 11.8 |
| 📍 | **22** | 0.153 | -0.812 | 0.305 | 13.5 |
| 📍 | **45** | 0.267 | -0.454 | 0.413 | 13.9 |
| ✖ | **81** | -0.129 | -0.221 | 0.423 | 12.4 (4th, not used) |

New row: $x_1$ = -0.100, $x_2$ = -0.643. Record 67: d = square root of ((-0.123 − (-0.100))² + (-0.770 − (-0.643))²) = 0.129. Prediction = (11.8 + 13.5 + 13.9) / 3 = 13.1 kWh/t. Actual: 12.6 kWh/t.

# The size of k matters: too small chases noise, too large flattens the pattern

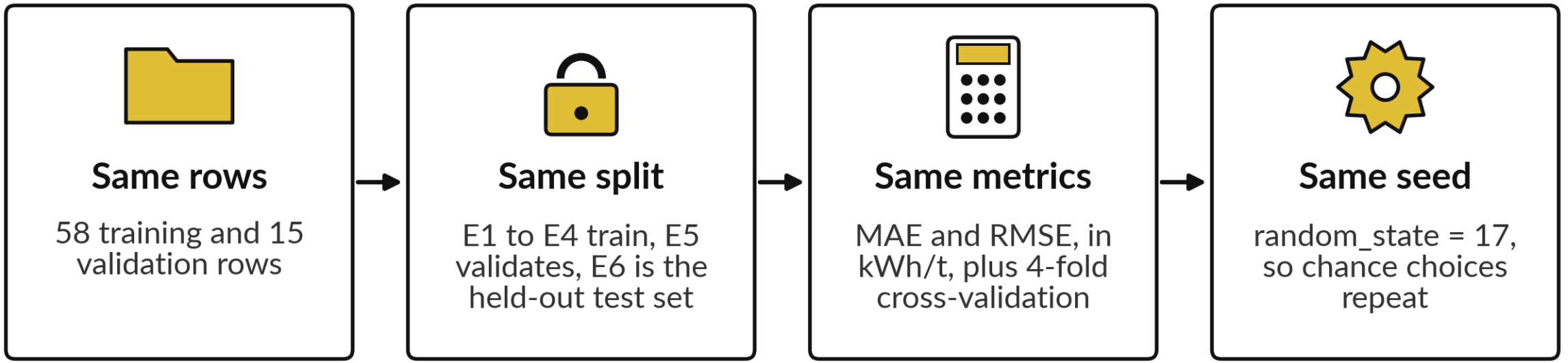



**k is a hyperparameter: a setting we choose before training. We try a few values and compare the validation MAE.**

Each bar is the k-NN validation MAE for one k. Shorter is better.

k = 1 copies one row, noise included. k = 20 averages so many rows that the pattern is lost.

# A fair model comparison keeps four things the same for every model




The picture shows the four rules of the protocol that every candidate model follows. Change one thing at a time: if two things change, you cannot say which one caused the difference.

# Every model gets MAE, RMSE and four-fold cross-validation scores

Validation MAE of model m

$$\mathrm{MAE}_m = \frac{1}{n}\sum_{i=1}^{n} |\, y_i - \hat{y}_{i,m} |$$

Validation RMSE of model m

$$\mathrm{RMSE}_m = \sqrt{\frac{1}{n}\sum_{i=1}^{n} (y_i - \hat{y}_{i,m})^2}$$

Cross-validated MAE

$$\mathrm{CV}_m = \frac{1}{4}\sum_{f=1}^{4} \mathrm{MAE}_{m,f}$$

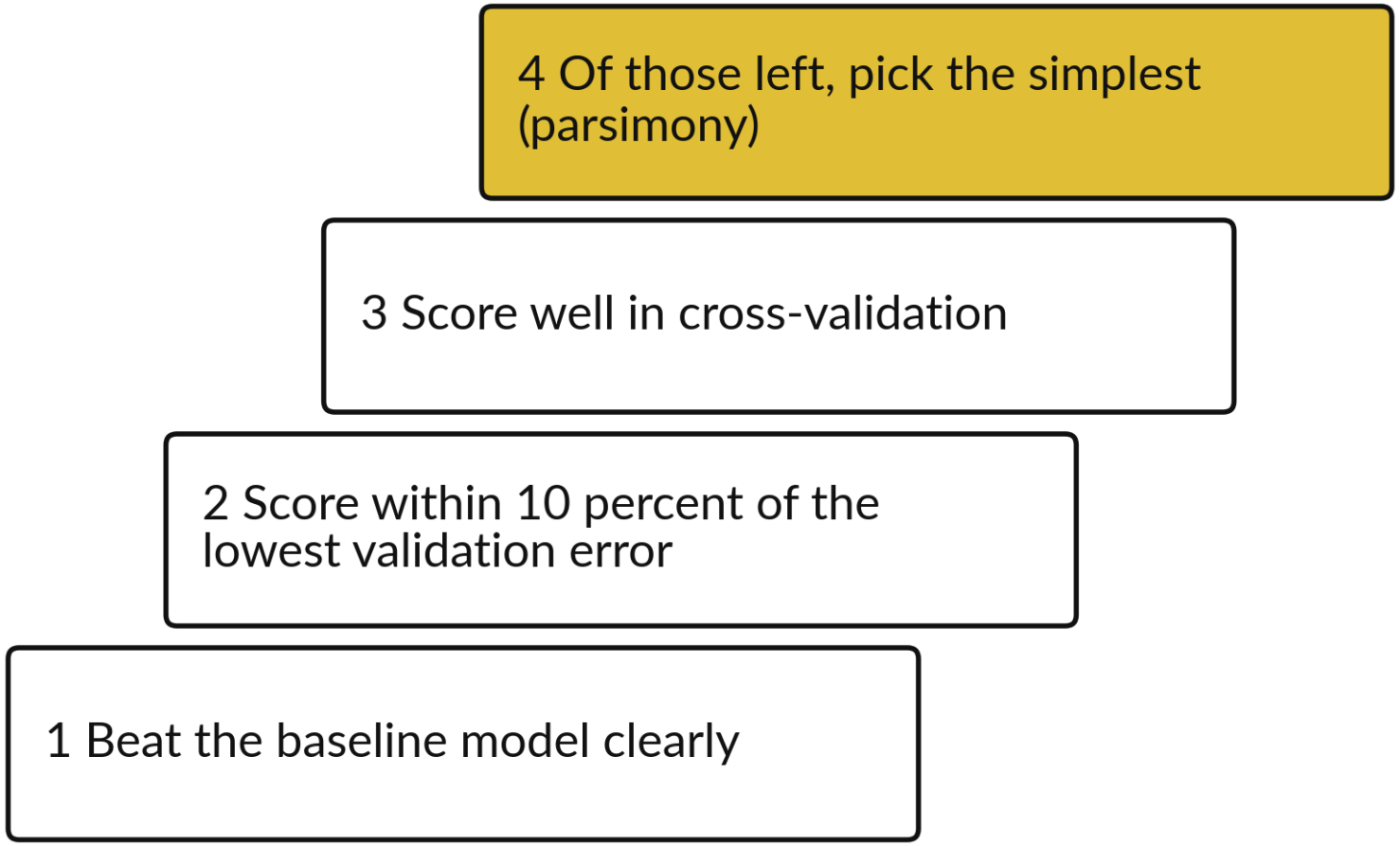
| Symbol | What it means | In our data |
|---|---|---|
| **m, i, n** | Model, validation row, number of rows | Five models; group E5, n = 15 |
| **y, ŷ** | Actual target, and prediction (kWh/t) | Linear regression: MAE = 0.67, RMSE = 0.81 |
| **MAE, RMSE** | Mean absolute error; root mean squared error | Same unit as the target (kWh/t) |
| **CV (model m)** | Cross-validated MAE: mean of 4 fold MAEs | Linear regression: 0.73, 0.57, 0.78, 0.72; mean 0.70 kWh/t |

# Model comparison table: five candidate models on the same 15 validation rows

| | Model | Validation MAE | Validation RMSE | Cross-validated MAE (mean of 4 folds) | Lowest to highest fold MAE |
|---|---|---|---|---|---|
|  | **Baseline model** | 2.71 | 3.28 | 2.39 | 1.89 to 2.72 |
|  | **Linear regression** | 0.67 | 0.81 | 0.70 | 0.57 to 0.78 |
|  | **Decision tree, depth 4** | 1.20 | 1.45 | 1.09 | 0.84 to 1.20 |
|  | **Ridge regression, alpha 1** | 0.70 | 0.84 | 0.71 | 0.59 to 0.76 |
|  | **k-NN, k = 3** | 0.63 | 0.82 | 0.85 | 0.71 to 1.04 |

Errors in kWh/t; lower is better. Baseline model: predicts the training mean. Ridge regression (two features) uses load_pct and ambient_c. A fold is one group, E1 to E4, held out.

# Model selection: choose the simplest model that is good enough (parsimony)




**Model selection: choose one of the candidate models. Parsimony: prefer the simpler model when scores are close.**

Baseline model: MAE 2.71 kWh/t. Linear regression: 0.67.

k-NN, k = 3 wins on validation (0.63 against 0.67), but linear regression wins in cross-validation (0.70 against 0.85).

# The model comparison table has three limits, and you must say them out loud



The data are synthetic, which means invented for teaching. Real plant data would behave differently.

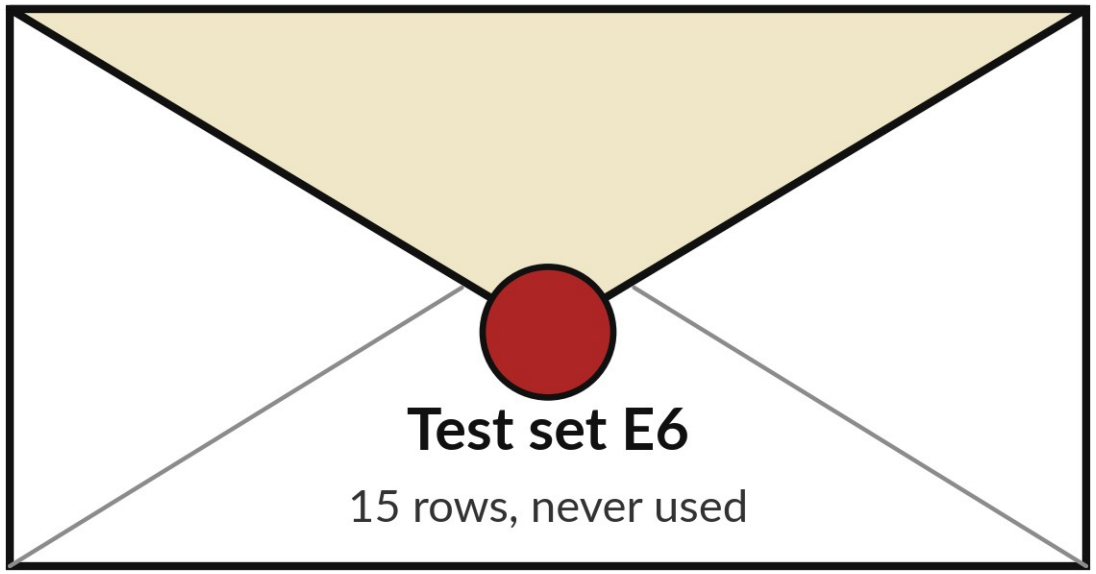


There are only 15 validation rows, all from group E5. A small gap between two models may be luck.



The test set E6 has 15 rows and no model has used them. We use it once, after we choose.

# The held-out test set is used once, after the model is chosen




Used once, after you choose

**The held-out test set (hold-out set) is drawn as a sealed envelope. No model has used its rows.**

It is the last check. We use it once, after we choose a model, for one honest score of generalisation.

Using it and then changing your choice makes it an unfair test.

# A decision memo fits on one page and has seven parts

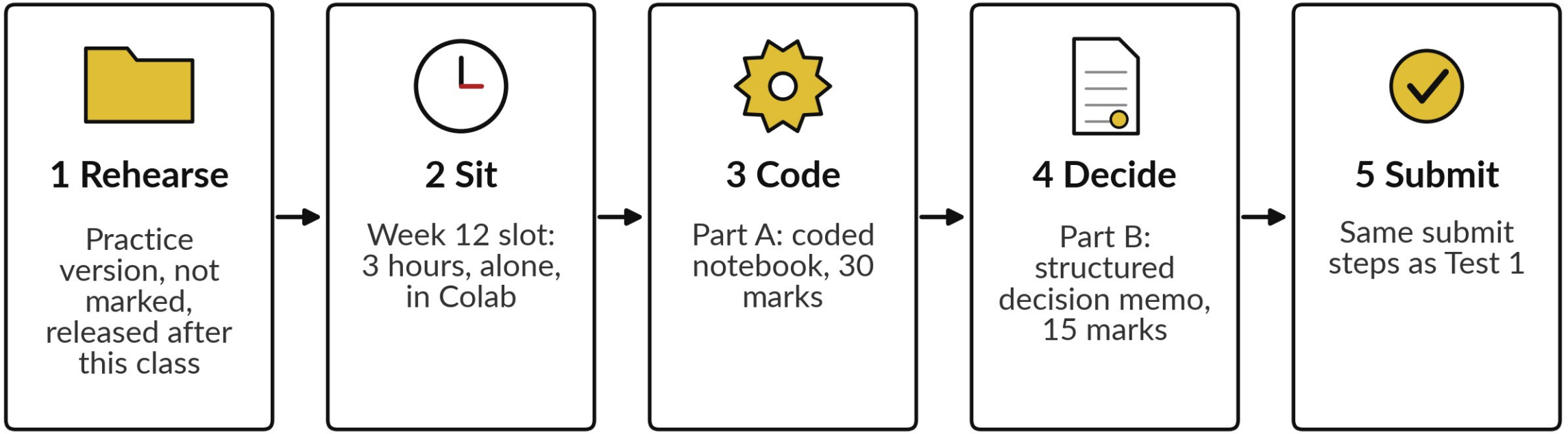
| Decision memo (example wording) | |
|---|---|
| 1 QUESTION | Can we predict crusher energy demand from load and temperature? |
| 2 DATA | 58 training rows and 15 validation rows; synthetic |
| 3 MODELS TRIED | Baseline, linear, tree, ridge, k-NN |
| 4 RESULT | Linear regression MAE 0.67 kWh/t; baseline 2.71 kWh/t |
| 5 LIMITS | Synthetic data; 15 validation rows; test set not yet used |
| 6 RECOMMENDATION | Use the linear regression model |
| 7 NEXT STEP | Score the held-out test set once and report it |

**A decision memo is a one-page note that lets the reader act. A related term is model card.**

The picture shows an example memo with its seven numbered parts, filled in with our numbers.

Give the result with its unit, say what you do not know, and end with what to do next.

# Your project is an individual, timed practical sitting that the computer grades




The five stages of the project: it is individual, timed and auto-graded, covers only Weeks 1 to 11, and is worth 45 percent. There is no demo, no presentation and no clinic.

# The project is auto-graded and worth 45 percent of the course

| Item | What you need to know |
|---|---|
| **Format** | Individual, timed, highly automated and auto-graded, like Practical Test 1. A 3-hour supervised practical sitting in Colab. It covers only Weeks 1 to 11. |
| **Part A: 30 marks** | The coded notebook, auto-graded. |
| **Part B: 15 marks** | A structured decision memo inside the notebook, auto-graded: structured fields cross-checked against your own results, plus keyword checks on two short answers. |

Nothing in the project is taught after Class 4.

# A rehearsal comes first, and the sitting is in your Week 12 class slot

| | Item | What you need to know |
|---|---|---|
|  | **Rehearsal** | A practice version of the project is released right after Class 4. It is not marked. Use it to practise the same steps. |
|  | **The sitting** | Proposed: Friday 4 December, 08:00 to 11:00 (lecturer to confirm). Your own Week 12 class slot. |
|  | **How to submit** | Restart and run all, save, then the submit cell, the same mechanism as Practical Test 1. The Practical Test 1 rule on AI and copying applies. |

All dates are proposed. Your lecturer confirms them. There is no demo, no presentation and no clinic.

# Read the recipe before you write it: a dictionary of models and a loop

```
models = {"linear": with_imputer(LinearRegression()),
          "tree": with_imputer(DecisionTreeRegressor(
              max_depth=4, random_state=17))}
for name, model in models.items():
    model.fit(X_train, y_train)
    y_pred = model.predict(X_valid)
    print(name, mean_absolute_error(y_valid, y_pred))
```

## In plain words

A dictionary stores each model under a name.

with_imputer is a helper, a small function we write once and reuse. It makes a scikit-learn pipeline: median imputation (SimpleImputer) of missing values, then the model.

The loop takes each model in turn: train (fit) on the training set, predict the validation set (inference), print the MAE.

To add a model, add one line to the dictionary.

# Today in five lines: test ideas, compare fairly, choose simply, write it down



A new feature is an idea: test it with a feature ablation.



Ridge regression and k-NN need scaled features.



A fair model comparison keeps rows, split, metrics and seed the same.



Choose the simplest model that is good enough (parsimony), and say the limits.



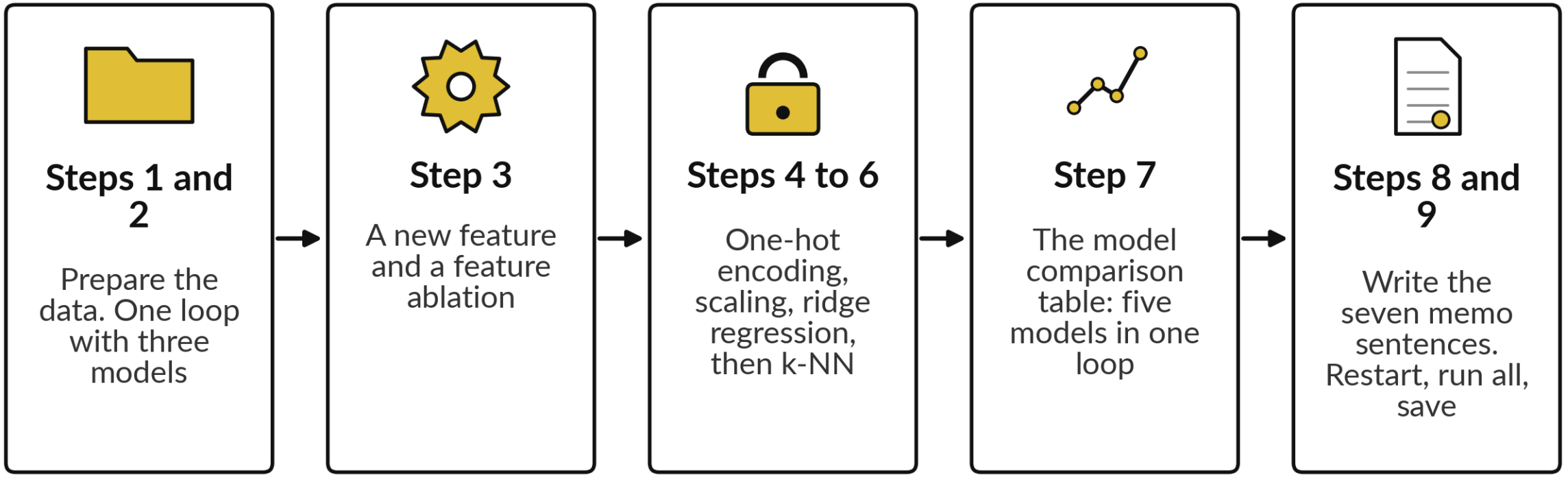
Use the held-out test set once, at the end, and write the decision memo.

KLAI

# Part 2: the lab

Close the slides. Open the notebook. One model comparison table, five candidate models, your own data.

# The lab has nine notebook steps in five groups, and the model comparison table is the centre




The nine notebook steps, in five groups. In each step: predict, run, then read.

# Check your numbers against these before you move on

| Step | You should see |
|---|---|
| **Training, validation, test rows** | 58, 15, 15 |
| **Linear regression, two features, validation MAE** | 0.67 kWh/t |
| **Linear regression plus load_x_ambient, validation MAE** | 0.67 kWh/t |
| **Linear regression with load_pct only, validation MAE** | 0.98 kWh/t |
| **Ridge regression, degree-6 polynomial of load_pct alone, alpha 1, validation MAE** | 0.99 kWh/t |
| **Ridge regression, two features, alpha 1, validation MAE (a different model)** | 0.70 kWh/t |
| **k-NN, k = 3, validation MAE** | 0.63 kWh/t |
| **Linear regression, cross-validated MAE (mean of 4 folds; lowest to highest)** | 0.70 (0.57 to 0.78) |

If your number differs, stop and ask. Do not change a number to match. Find the step that went wrong.

# Four errors you will meet, what they mean and what to do

| | Error message | What it says | What to do |
|---|---|---|---|
|  | **KeyError: load_x_ambient** | The new feature exists in one table only | Build it on both the training table and the validation table |
|  | **ValueError: contains NaN** | A model received a missing value (NaN) and has no imputation step | Wrap the model with with_imputer(...) or add a SimpleImputer to the pipeline |
|  | **inconsistent numbers of samples** | X and y have different numbers of rows because they come from different tables | Rebuild X_train and y_train from the same table |
|  | **NameError: with_imputer** | The cell that defines the helper function with_imputer has not been run | Run all cells from the top, in order |

# How this class counts towards your coursework



The Block 2 quiz opens in the last 20 minutes of this class and stays open for 24 hours.



The Block 2 assignment (A9, A10, A11) closes on Friday 4 December, 08:00 (Kampala time).



The Engineering ML project is an individual, timed, auto-graded 3-hour sitting worth 45 percent (Part A 30, Part B 15). Proposed: Friday 4 December, 08:00 to 11:00, lecturer to confirm.



The rehearsal (a practice project) is released after this class and is not marked.

# Save the notebook and keep your memo for the project

## What you keep from today

Your lab notebook, saved with all cells run in order, and your decision memo with its sentences filled in.

### Our question today

*Which model should we recommend for predicting crusher energy demand, and how do we explain the choice to someone who pays?*

### Next class

Week 12 (proposed): the timed Engineering ML project, 3 hours in your usual class slot. A rehearsal version is released after this class.

**Take away**

**Saved notebook**
*Restart and run all, then save*

**Model comparison table**
*Five candidate models, one protocol*

**Decision memo**
*Seven parts, one page*

**Next**
*Week 12: project sitting*